SKILL – 1
GIT VERSION CONTROL

2400030888.T. Sasi Bhaskar Kumar

Introduction :

Git Version Control is used to manage and track changes in project files. It helps developers work together by saving different versions of code safely. Using Git, we can create branches, make updates, and merge changes without losing data. This skill helps us understand how real software teams collaborate and manage their projects efficiently

TASKS :

STEP 1 : Create a new project folder, initialize Git using git init, and configure Git with your username and email.

1. Open Git Bash or Terminal

1. **Create a new folder**

```
bash

mkdir git-skill1
cd git-skill1
```

2. **Initialize Git**

```
bash

git init
```

3. **Configure username and email**

```bash
git config --global user.name "Your Name"
git config --global user.email "yourmail@gmail.com"
```

4. **Verify configuration**

```bash
git config --list
```

STEP 2: Create two files (example: main.java, notes.txt) with sample content, check the repository status, and add them to staging.

1. **Create two files**

```bash
touch main.java notes.txt
```

2. **Add sample content**

```bash
echo "Hello Git" > main.java
echo "Git Skill 1 notes" > notes.txt
```

3. **Check repository status**

```bash
git status
```

✔ **Files will appear in red (untracked).**

4. **Add files to staging**

```bash
git add .
```

5. **Check status again**

```bash
git status
```

✔ **Files will appear in green (staged).**

STEP 3: Commit the staged files with a meaningful commit message

```bash
git commit -m "Initial commit with main.java and notes.txt"
```

✔ **Files are now saved in Git history.**

STEP 4: Create a new repository on GitHub, connect your local repo using the remote URL and push the committed changes.

1. Go to GitHub → New Repository

- **Repository name: git-skill1**
- **Keep it Public**
- ❌ **Do NOT add README**

2. Copy repository URL

3. Connect local repo to GitHub

```bash
git remote add origin https://github.com/username/git-skill1.git
```

4. Push code

```bash
git branch -M main
git push -u origin main
```

✔ **Code is now visible on GitHub.**

STEP 5: Create the first branch (example: feature-update), make modifications, and add + commit the changes.

1. **Create and switch branch**

```
bash

git checkout -b feature-update
```

2. **Modify file**

```
bash

echo "Feature update added" >> main.java
```

3. **Add and commit**

```
bash

git add main.java
git commit -m "Added feature update"
```

STEP 6️⃣: Create a second branch (example: bug-fix), apply changes, and add + commit them.

1. **Switch to main**

```
bash

git checkout main
```

2. **Create new branch**

```
bash

git checkout -b bug-fix
```

3. **Modify file**

```
bash

echo "Bug fixed here" >> main.java
```

4. **Add and commit**

```
bash

git add main.java
git commit -m "Bug fix applied"
```

STEP 7: Switch back to the main branch and merge both branches one after the other

1. **Switch to main**

```bash
git checkout main
```

2. **Merge feature branch**

```bash
git merge feature-update
```

3. **Merge bug-fix branch**

```bash
git merge bug-fix
```

✔ **If no conflict → merge completes.**

STEP 8: If any merge conflict occurs, resolve it manually, commit the fix, and complete the merge

1. **Open conflicted file**

```text
<<<<<<< HEAD
Main content
=======
Branch content
>>>>>>> bug-fix
```

2. **Edit manually and keep correct code ((Delete conflict symbols)**

3. **Add resolved file**

```bash
git add main.java
```

4. **Commit resolution**

```bash
git commit -m "Resolved merge conflict"
```

✅ FINAL CHECK

```bash
git log --oneline --graph
```

✔ **Shows all commits and merges.**